# myKaria: a production AI product built and run by one developer

Oluwatobi Lukan | Hamburg, Germany | Full-stack engineer, AI application pipelines

myKaria turns a job ad into a tailored CV and cover letter that a candidate can actually defend in the interview. I designed it, built it and run it: database, backend, frontend, AI pipeline, billing and deployment. Around 200 commits, 68 migrations, 23 document templates, live in production. It began as my own tool during my own job search in Germany.

## THE PROBLEM, STATED PRECISELY

AI CV tools optimise for interview invitations, so they exaggerate. The candidate gets the interview and loses it, because the document claims things they cannot back up. Recruiters now discount generated CVs by default. So defensibility is not a writing problem, it is a ranking and policy problem: which skills genuinely match, at what level they can honestly be claimed, which ones to feature or omit, and how to cover missing requirements with real adjacent experience rather than invention.

## THREE DECISIONS THAT SHAPED THE BUILD

- **One matching call, not three.** The obvious pipeline is match, plan, write. I collapsed it to a single indexed matching call whose result feeds both documents. That removed a third of the latency and removed a class of bug where the CV and the letter disagreed about the same skill.
- **The model judges, the code enforces policy.** Anything that must hold every time, such as a beginner skill never appearing as expert, is implemented in tested code rather than requested in a prompt. Truncated model output is treated as a correctness failure, because an empty skill list used to let anything through.
- **A modular monolith.** At one developer, splitting services adds operational cost and buys nothing. The useful boundaries are module boundaries: 33 route modules, domain services, their own tests.

## EVIDENCE

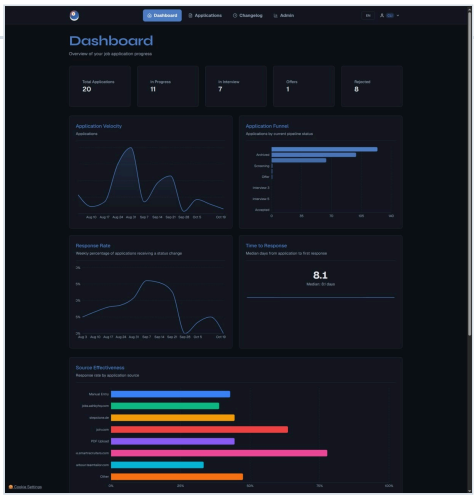
- 148 unit tests in the public slice covering prompt assembly and the matching pipeline end to end, including failure paths
- 274 test and spec files across the full repository
- Provider-agnostic LLM layer: OpenAI, Anthropic, Ollama, OpenRouter and any OpenAI-compatible endpoint, with retry, fallback and per-call cost resolution
- Multi-tenant mechanics: JWT auth with email verification, tier entitlements, Stripe billing, data isolation, retention and deletion jobs
- Docker Compose environments, Jenkins pipeline, Nginx-backed deployment

## WHERE IT CAME FROM

myKaria started as my own tool. I was applying for job after job in Germany, tailoring every CV and letter by hand: reading the company, judging which of my skills genuinely mattered for the role, rewriting the same history twenty times. So I built the pipeline for myself.

Then I used it on my own applications. Company research, honest skill matching, generated documents, section by section edits before sending. What survived that use is what shipped: anything that did not get me a better application out the door did not make it into the product. Once it was working reliably in my own workflow, making it available to other people was the obvious next step.

That origin explains the codebase. The honest skill tiers exist because I would not walk into an interview unable to defend my own CV. Section editing exists because a generated document always needs one more pass.




Application tracker: velocity, funnel by stage, weekly response rate, median time to first response, response rate by source.

## WHAT THIS SAYS ABOUT ME AS AN ENGINEER

I can take an ambiguous product idea to a running system: schema design, provider abstraction, prompt engineering with guardrails, billing, deployment, and the unglamorous parts such as retention jobs and idempotency.

Product mykaria.com | Code github.com/ztobs/mykaria-showcase | Architecture notes in *docs/architecture.md* |
Contact: tobilukan@gmail.com